



RAPPORT DE PROJET

SYSTÈME DE GESTION DE RESERVATION D'HÔTEL

MOHAMED YAZID REBEI

RAYEN BOUJILA



INTRODUCTION

CETTE APPLICATION CENTRALISE LA GESTION DES HÔTELS VIA UNE BASE DE DONNÉES RELATIONNELLE ET UNE INTERFACE INTUITIVE. ELLE OPTIMISE QUATRE AXES CLÉS :

1. GESTION DES CHAMBRES

- AJOUT/MODIFICATION DES DÉTAILS (TYPE, CAPACITÉ, ÉTAT : LIBRE, OCCUPÉE, EN RÉNOVATION).

2. GESTION DES CLIENTS

- ENREGISTREMENT, SUIVI PERSONNALISÉ ET IDENTIFICATION DES CLIENTS FIDÈLES.

3. RÉSERVATIONS INTELLIGENTES

- CRÉATION/MODIFICATION/ANNULATION AVEC VÉRIFICATION AUTOMATIQUE DES DISPONIBILITÉS POUR ÉVITER LES CONFLITS.

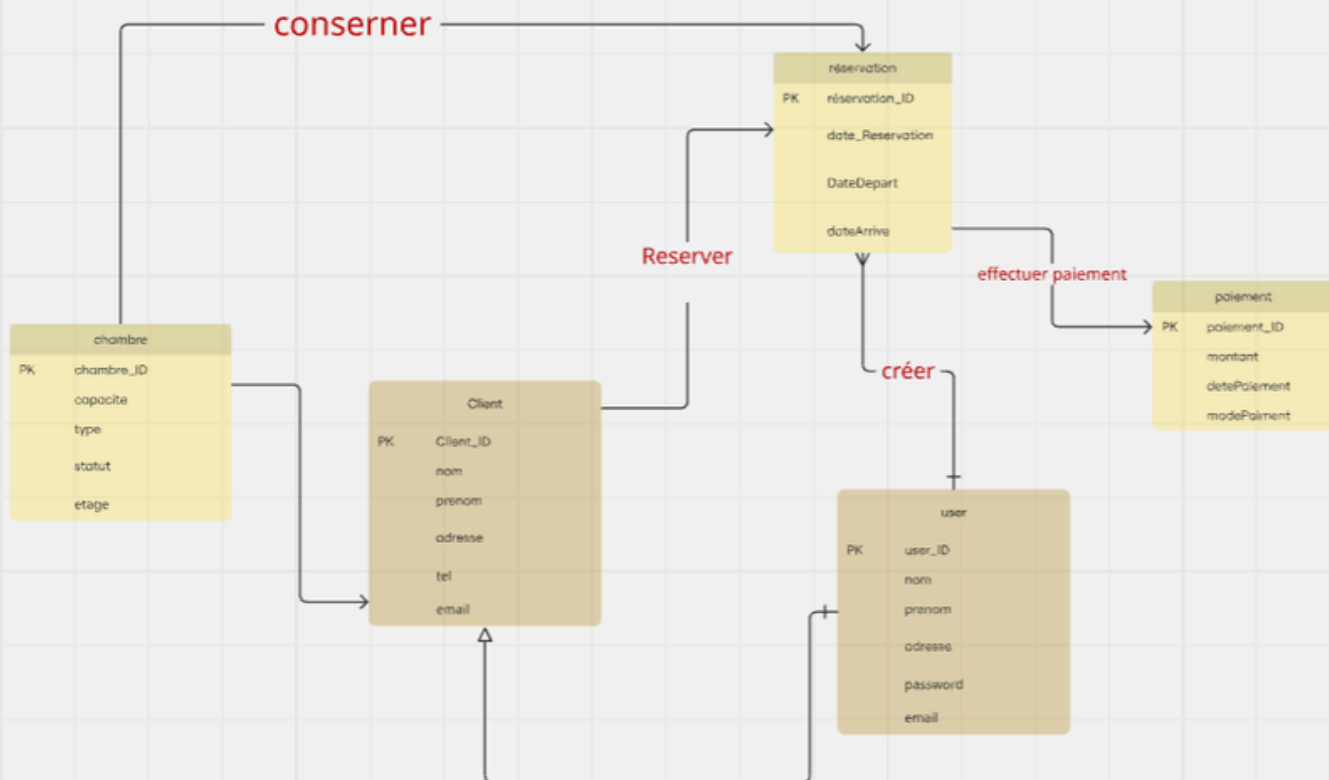
4. ANALYSE STRATÉGIQUE

- CONSULTATION DES CHAMBRES DISPONIBLES/OCCUPÉES, DÉTECTION DES CHAMBRES POPULAIRES ET CLIENTS RÉGULIERS.

FONCTION AVANCÉE : UNE PROCÉDURE PL/SQL CALCULE LES CHAMBRES DISPONIBLES POUR UNE PÉRIODE ET UN NOMBRE DE PERSONNES DONNÉS, EN TEMPS RÉEL

2. Spécifications Fonctionnelles

MODÈLE ENTITÉ ASSOCIATION :



MODÈLE RELATIONNEL :

Chambre (numChambre, capacite, typeChambre, statut, etage)

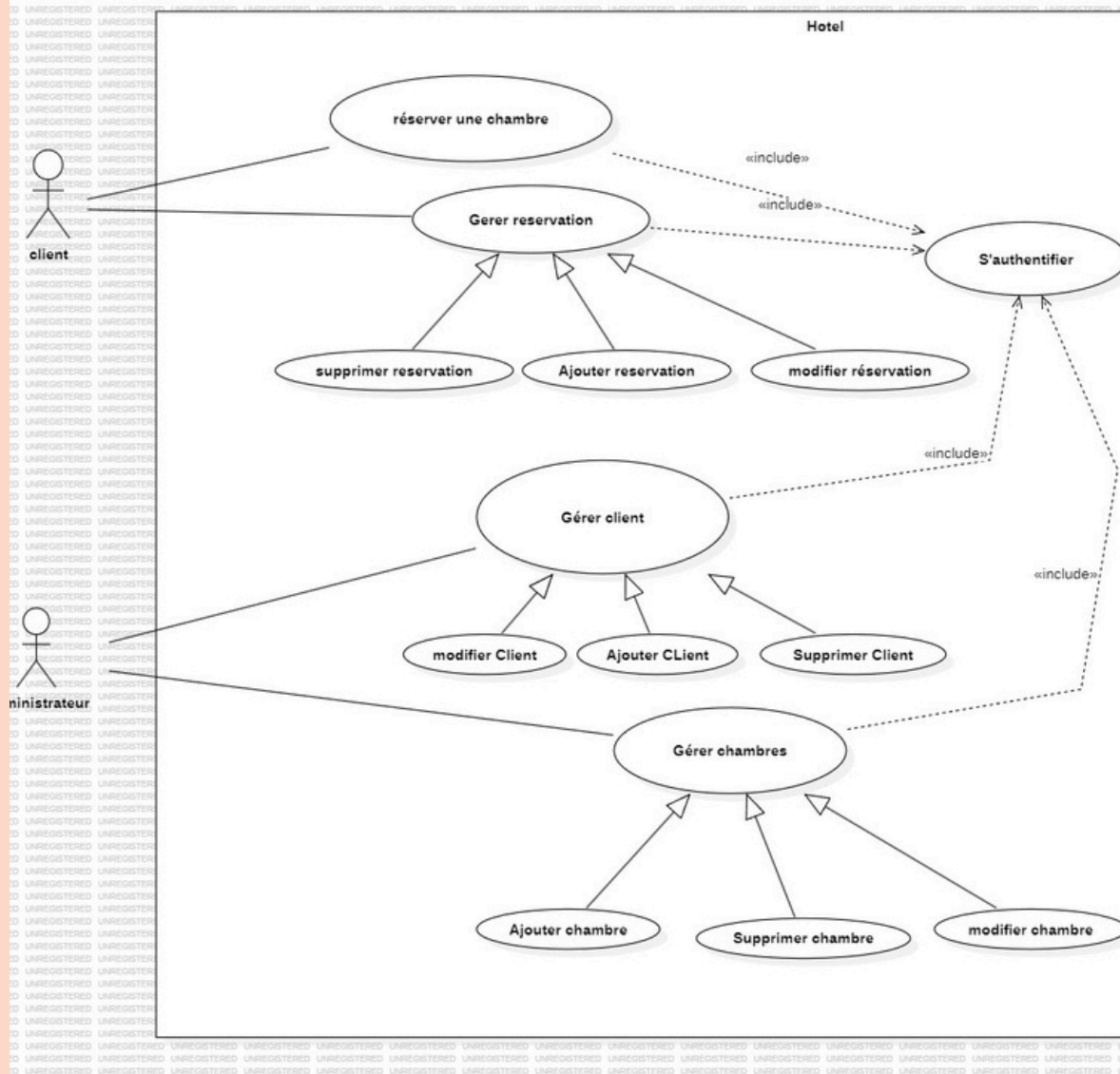
Reservation (idReservation, dateReservation, modeReservation, dateArrivee, dateDepart, commentaireClient, #numChambre)

Paiement (idPaiement, montant, datePaiement, fraisSupplementaire, modePaiement, #idReservation)

Client (idClient, nom, prenom, adresse, telephone, email)

Utilisateur (idUser, identifiant, motDePasse, roleUser, #idClient)

DIAGRAMME DE CAS D'UTILISATION :



MODÉLISATION DES CAS D'UTILISATION:

Le diagramme de cas d'utilisation est un outil fondamental dans la phase de modélisation fonctionnelle d'un système. Il permet de visualiser, de manière claire et synthétique, les interactions possibles entre les acteurs externes (utilisateurs) et le système étudié, en l'occurrence ici, une application de gestion d'hôtel. Chaque cas d'utilisation représente un scénario fonctionnel attendu, sans pour autant définir les aspects techniques de l'implémentation.

Acteurs identifiés :

- **Administrateur** : responsable de la gestion globale de l'application (utilisateurs, chambres, clients...).
- **Réceptionniste** : gère les réservations, les clients et les paiements.
- **Client** : effectue des recherches et peut éventuellement effectuer des réservations (si prévu dans le système).

CAS D'UTILISATION PRINCIPAUX :

Authentification

- **Acteurs** : Administrateur, Réceptionniste
- **Description** : Permet aux utilisateurs de s'authentifier pour accéder à leurs interfaces respectives via un identifiant et un mot de passe.
- **Objectif** : Sécuriser l'accès aux fonctionnalités.

Effectuer une recherche

- **Acteurs** : Client, Réceptionniste
- **Description** : Recherche de chambres selon des critères (mot-clé, date, capacité...).
- **Objectif** : Trouver rapidement des chambres disponibles.
-

CAS D'UTILISATION PRINCIPAUX :

Gestion des utilisateurs

- **Acteur** : Administrateur
- **Description** : Ajouter, modifier ou supprimer des comptes utilisateurs
- **Objectif** : Contrôler les accès au système.
-

Gestion des clients

- **Acteurs** : Réceptionniste, Administrateur
- **Description** : Ajouter, modifier ou supprimer les données clients.
- **Objectif** : Maintenir une base de données client à jour.
-

CAS D'UTILISATION PRINCIPAUX :

Gestion des factures / paiements

- **Acteurs** : Réceptionniste, Administrateur
- **Description** : Générer, modifier ou supprimer des paiements associés aux réservations.
- **Objectif** : Assurer le suivi financier des réservations.
-

Gestion des réservations

- **Acteurs** : Réceptionniste, Administrateur
- **Description** : Ajouter, modifier ou annuler des réservations.
- **Objectif** : Organiser efficacement les séjours clients.
-

CAS D'UTILISATION PRINCIPAUX :

Gestion des chambres

- **Acteur** : Administrateur
- **Description** : Ajouter, modifier ou supprimer les chambres de l'hôtel.
- **Objectif** : Gérer l'offre d'hébergement de l'établissement.
-

Ce **diagramme** met en évidence les fonctionnalités clés attendues d'un système de gestion d'hôtel et leur répartition selon les rôles des **utilisateurs**. L'approche orientée cas d'utilisation permet de mieux cerner les besoins fonctionnels sans entrer dans les détails techniques. Chaque fonctionnalité se traduit par un cas d'utilisation distinct, centré sur un objectif métier clair.

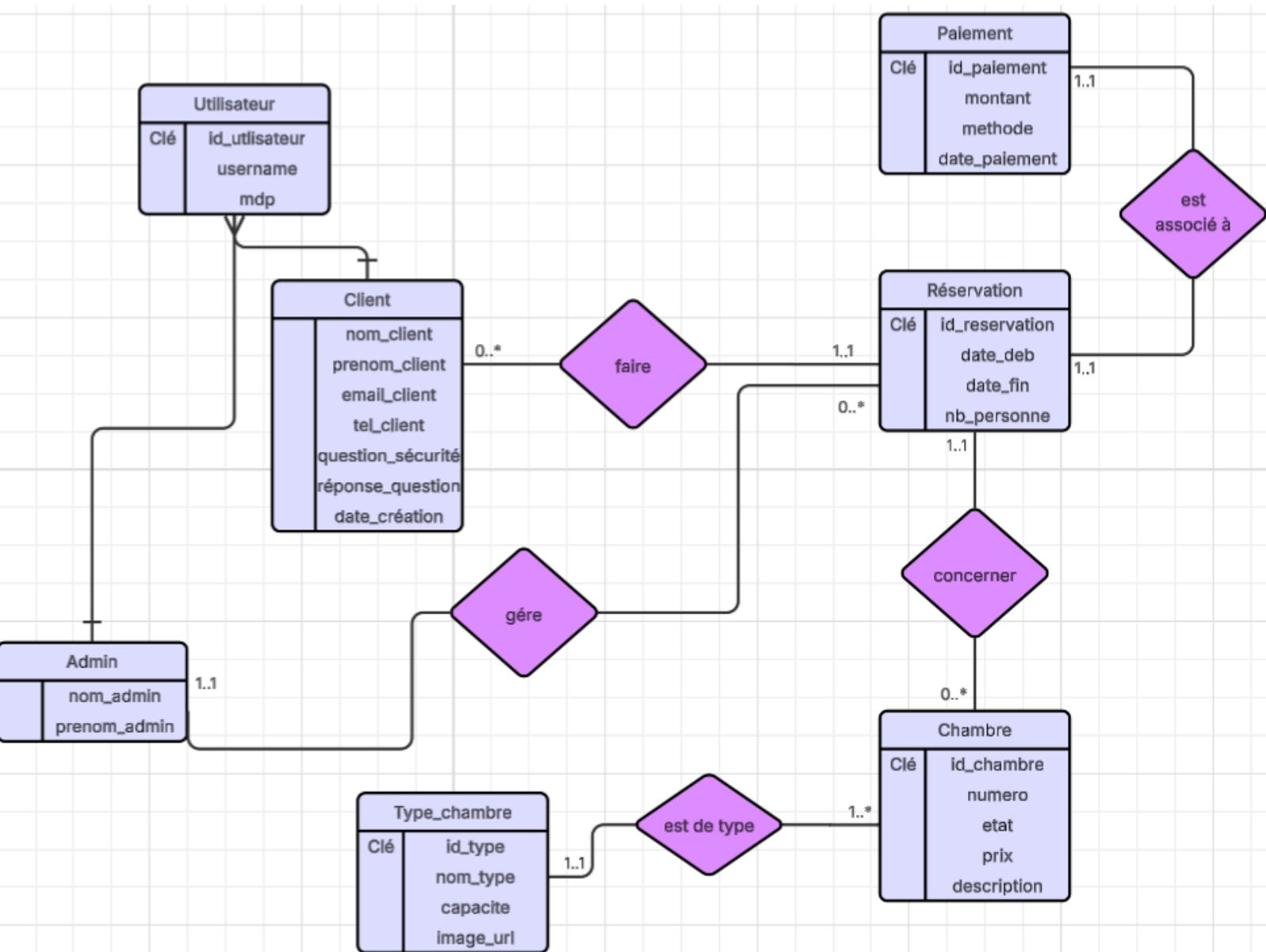
DIAGRAMME DE CLASSE :

Ce diagramme représente les entités principales du système de gestion d'un hôtel, ainsi que les relations entre elles. Il modélise la structure des données que l'application devra gérer dans sa base de données.

Ce diagramme est essentiel car :

- Il définit la structure de la base de données.
- Il assure la cohérence et l'intégrité des données.
- Il facilite le développement backend en guidant la conception des tables SQL.

PRESENTATION DU DIAGRAMME:





PROGRAMMATION



CONNEXION à LA BASE DE DONNÉES ORACLE :

```
package hotelmanagement;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {

    private static final String DB_URL =
        "jdbc:oracle:thin:@localhost:1521/freexdb";
    private static final String DB_USER = "SYS as SYSDBA";
    private static final String DB_PASSWORD = "rayen2004";

    public static Connection getConnection() throws
        SQLException {
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            java.util.Properties props = new
                java.util.Properties();
            props.put("user", DB_USER);
            props.put("password", DB_PASSWORD);
            props.put("internal_logon", "sysdba");

            return DriverManager.getConnection(DB_URL, props);
        } catch (ClassNotFoundException e) {
            throw new SQLException("Oracle JDBC Driver not
                found", e);
        }
    }
}
```

CONNEXION à LA BASE DE DONNÉES ORACLE :

1

Charger le pilote JDBC Oracle

2

Définir les informations de connexion (URL, utilisateur, mot de passe)

3

Configurer les propriétés pour la connexion en mode administrateur (SYSDBA)



CONNEXION à LA BASE DE DONNÉES ORACLE :

4

Établir la connexion avec la base de données

5

Retourner l'objet de connexion pour l'utiliser dans l'application

6

Gérer les erreurs en cas d'échec de connexion.

FONCTIONS POUR GÉRER LES CHAMBRES :

CHARGER DE CHAMBRE :

```
private void loadRoomsData(DefaultTableModel model) {  
    try (Statement stmt = connection.createStatement();  
        ResultSet rs = stmt.executeQuery("SELECT * FROM  
rooms ORDER BY room_id")) {  
  
        model.setRowCount(0);  
  
        while (rs.next()) {  
            model.addRow(new Object[]{  
                rs.getString("room_id"),  
                rs.getString("room_type"),  
                String.format("%.2f",  
rs.getDouble("price")),  
                rs.getString("status"),  
                "Modifier",  
                "Supprimer"  
            });  
        }  
    } catch (SQLException e) {  
        showError("Erreur lors du chargement des chambres:  
" + e.getMessage());  
    }  
}
```

MODIFICATION DE CHAMBRE :

```
private void editRoom(int row) {
    String roomId = (String) tableModel.getValueAt(row, 0);

    try (PreparedStatement pstmt =
        connection.prepareStatement(
            "SELECT * FROM rooms WHERE room_id = ?")) {

        pstmt.setString(1, roomId);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            String currentType = rs.getString("room_type");
            double currentPrice = rs.getDouble("price");
            String currentStatus = rs.getString("status");
            try (PreparedStatement updateStmt =
                connection.prepareStatement(
                    "UPDATE rooms SET room_type = ?, price = ?,
                    status = ? WHERE room_id = ?")) {

                updateStmt.setString(1, "Nouveau type");
                updateStmt.setDouble(2, 150.0);
                updateStmt.setString(3, "Nouveau statut");
                updateStmt.setString(4, roomId);
                updateStmt.executeUpdate();

                loadRoomsData(tableModel);
            }
        }
    } catch (SQLException ex) {
        showError("Erreur: " + ex.getMessage());
    }
}
```

SUPPRIMER DE CHAMBRE :

```
private void deleteRoom(int row) {
    String roomId = (String) tableModel.getValueAt(row, 0);

    try {
        try (PreparedStatement statusStmt =
            connection.prepareStatement(
                "SELECT status FROM rooms WHERE room_id = ?")) {

            statusStmt.setString(1, roomId);
            ResultSet rs = statusStmt.executeQuery();
            if (rs.next() &&
                !"Available".equalsIgnoreCase(rs.getString("status"))) {
                throw new SQLException("Chambre non
disponible");
            }
        }
        try (PreparedStatement checkStmt =
            connection.prepareStatement(
                "SELECT COUNT(*) FROM reservations WHERE room_id =
? AND status != 'Cancelled'")) {

            checkStmt.setString(1, roomId);
            ResultSet rs = checkStmt.executeQuery();
            if (rs.next() && rs.getInt(1) > 0) {
                throw new SQLException("Réservations
actives");
            }
        }
        try (PreparedStatement deleteStmt =
            connection.prepareStatement(
                "DELETE FROM rooms WHERE room_id = ?")) {

            deleteStmt.setString(1, roomId);
            deleteStmt.executeUpdate();
            tableModel.removeRow(row);
        }
    } catch (SQLException ex) {
        showError("Erreur: " + ex.getMessage());
    }
}
```

FONCTIONS POUR GÉRER LES RÉSERVATIONS :

CHARGER DE RESERVATION :

```
private void loadReservationsData(DefaultTableModel model) {
    try (Statement stmt = connection.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM
reservations ORDER BY reservation_id")) {

        model.setRowCount(0);

        SimpleDateFormat dateFormat = new
SimpleDateFormat("yyyy-MM-dd");

        while (rs.next()) {
            String status = rs.getString("status");
            String confirmAction =
"Pending".equalsIgnoreCase(status) ? "Confirmer" : "";
            String cancelAction =
("Pending".equalsIgnoreCase(status) ||
"Confirmed".equalsIgnoreCase(status)) ? "Annuler" : "";

            model.addRow(new Object[]{
                rs.getString("reservation_id"),
                rs.getString("guest_name"),
                rs.getString("room_id"),
                dateFormat.format(rs.getDate("check_in")),

                dateFormat.format(rs.getDate("check_out")),
                status,
                confirmAction,
                cancelAction
            });
        }
    } catch (SQLException e) {
        showError("Erreur lors du chargement des
réservations: " + e.getMessage());
    }
}
```

MODIFIER DE RESERVATION :

```
private void loadReservationsData(DefaultTableModel model) {
    try (Statement stmt = connection.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM
reservations ORDER BY reservation_id")) {

        model.setRowCount(0);

        SimpleDateFormat dateFormat = new
SimpleDateFormat("yyyy-MM-dd");

        while (rs.next()) {
            String status = rs.getString("status");
            String confirmAction =
"Pending".equalsIgnoreCase(status) ? "Confirmer" : "";
            String cancelAction =
("Pending".equalsIgnoreCase(status) ||
"Confirmed".equalsIgnoreCase(status)) ? "Annuler" : "";

            model.addRow(new Object[]{
                rs.getString("reservation_id"),
                rs.getString("guest_name"),
                rs.getString("room_id"),
                dateFormat.format(rs.getDate("check_in")),

                dateFormat.format(rs.getDate("check_out")),
                status,
                confirmAction,
                cancelAction
            });
        }
    } catch (SQLException e) {
        showError("Erreur lors du chargement des
réservations: " + e.getMessage());
    }
}
```

SUPPRIMER DE RESERVATION :

```
public boolean deleteReservation(String reservationId) throws
SQLException {
    Connection connection = null;
    try {
        connection = DatabaseConnection.getConnection();
        connection.setAutoCommit(false);

        // Vérifier le statut de la réservation
        String checkSql = "SELECT status FROM reservations WHERE
reservation_id = ?";
        try (PreparedStatement checkStmt =
connection.prepareStatement(checkSql)) {
            checkStmt.setString(1, reservationId);
            ResultSet rs = checkStmt.executeQuery();

            if (!rs.next()) {
                throw new SQLException("Réservation non
trouvée");
            }

            String status = rs.getString("status");
            if (!"Cancelled".equalsIgnoreCase(status)) {
                throw new IllegalStateException("Suppression
autorisée uniquement pour les réservations annulées");
            }
        }
    }
}
```

SUPPRIMER DE RESERVATION :

```
// Supprimer la réservation
String deleteSql = "DELETE FROM reservations WHERE
reservation_id = ?";
try (PreparedStatement deleteStmt =
connection.prepareStatement(deleteSql)) {
    deleteStmt.setString(1, reservationId);
    int affected = deleteStmt.executeUpdate();
    connection.commit();
    return affected > 0; // Retourne true si suppression
réussie
}
} catch (SQLException | IllegalStateException e) {
    if (connection != null) connection.rollback();
    throw e; // Propagation de l'exception
} finally {
    if (connection != null) connection.close();
}
}
```

FONCTIONS POUR GÉRER LES RÉSERVATIONS USER : CHARGER DE RESERVATION USER :

```
private void loadUserReservations(DefaultTableModel model) {  
    try (PreparedStatement pstmt =  
connection.prepareStatement(  
        "SELECT * FROM reservations WHERE guest_name =  
? ORDER BY check_in")) {  
  
        pstmt.setString(1, currentUser.getFullName());  
        ResultSet rs = pstmt.executeQuery();  
  
        model.setRowCount(0);  
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-  
MM-dd");  
  
        while (rs.next()) {  
            model.addRow(new Object[]{  
                rs.getString("reservation_id"),  
                rs.getString("room_id"),  
                sdf.format(rs.getDate("check_in")),  
                sdf.format(rs.getDate("check_out")),  
                rs.getString("status")  
            });  
        }  
    } catch (SQLException e) {  
        JOptionPane.showMessageDialog(this, "Error loading  
reservations: " + e.getMessage());  
    }  
}
```

LOGIN /LOGOUT USER :

```
private static void login(Scanner scanner) {
    System.out.println("\n=== Login ===");
    System.out.print("Username: ");
    String username = scanner.nextLine();

    System.out.print("Password: ");
    String password = scanner.nextLine();

    if(username.isBlank() || password.isBlank()) {
        System.out.println("Error: Username and password
required!");
        return;
    }

    User user = AuthService.authenticate(username,
password);
    if(user != null) {
        System.out.println("Login successful! Welcome " +
user.getFullName());
    } else {
        System.out.println("Error: Invalid credentials!");
    }
}
```



```
private static void signUp(Scanner scanner) {
    System.out.println("\n=== Sign Up ===");
    System.out.print("Full name: ");
    String fullName = scanner.nextLine();

    System.out.print("Username: ");
    String username = scanner.nextLine();

    System.out.print("Password: ");
    String password = scanner.nextLine();

    System.out.print("Confirm password: ");
    String confirmPassword = scanner.nextLine();

    if(!password.equals(confirmPassword)) {
        System.out.println("Error: Passwords don't match!");
        return;
    }

    if(AuthService.registerUser(fullName, username,
password)) {
        System.out.println("Account created successfully!");
    } else {
        System.out.println("Error: Username already
exists!");
    }
}
```

LOGOUT :



```
private void logout() {  
    try {  
        if (connection != null && !connection.isClosed()) {  
            connection.close();  
        }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    this.dispose();  
    new Login().setVisible(true);  
}
```



AUTHENTICATION :

```
public class AuthService {
    public static User authenticate(String username, String
password) {
        String sql = "SELECT username, password, role, full_name
FROM users WHERE username = ? AND password = ?";

        try (Connection conn =
DatabaseConnection.getConnection();
            PreparedStatement stmt =
conn.prepareStatement(sql)) {

            stmt.setString(1, username);
            stmt.setString(2, password);

            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) {
                    return new User(
                        rs.getString("username"),
                        rs.getString("password"),
                        rs.getString("role"),
                        rs.getString("full_name")
                    );
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```

AUTHENTICATION :

```
public static boolean registerUser(String fullName, String
username, String password) {
    String sql = "INSERT INTO users (username, password,
role, full_name) VALUES (?, ?, 'client', ?)";

    try (Connection conn =
DatabaseConnection.getConnection();
        PreparedStatement stmt =
conn.prepareStatement(sql)) {

        stmt.setString(1, username);
        stmt.setString(2, password);
        stmt.setString(3, fullName);

        int rowsInserted = stmt.executeUpdate();
        return rowsInserted > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
```



BASE DE DONNÉES : PL_SQL

CREATION DES TABLEAUX :

```
-- Table des utilisateurs (pour User.java)
CREATE TABLE users (
    username VARCHAR2(50) PRIMARY KEY,
    password VARCHAR2(100),
    role VARCHAR2(20),
    full_name VARCHAR2(100)
);

-- Table des chambres (rooms)
CREATE TABLE rooms (
    room_id VARCHAR2(20) PRIMARY KEY,
    room_type VARCHAR2(50),
    price NUMBER(10,2),
    status VARCHAR2(20) CHECK (status IN ('Available',
'Reserved', 'Maintenance', 'Pending'))
);

-- Table des réservations (reservations)
CREATE TABLE reservations (
    reservation_id VARCHAR2(50) PRIMARY KEY,
    guest_name VARCHAR2(100),
    room_id VARCHAR2(20),
    check_in DATE,
    check_out DATE,
    status VARCHAR2(20) CHECK (status IN ('Pending',
'Confirmed', 'Cancelled')),
    CONSTRAINT fk_room FOREIGN KEY (room_id) REFERENCES
rooms(room_id)
);
```

FONCTIONS PL-SQL:

```
-- Taux d'Occupation
CREATE OR REPLACE FUNCTION calculer_taux_occupation RETURN
VARCHAR2
AS
    taux_occupation NUMBER;
    total_chambres NUMBER;
    chambres_occupees NUMBER;
BEGIN
    -- Compter le nombre total de chambres
    SELECT COUNT(*) INTO total_chambres FROM Chambre;

    -- Compter les chambres réservées
    SELECT COUNT(*) INTO chambres_occupees
    FROM Chambre
    WHERE etat = 'Réservée';

    -- Calculer le taux d'occupation
    IF total_chambres > 0 THEN
        taux_occupation := (chambres_occupees * 100.0) /
total_chambres;
    ELSE
        taux_occupation := 0;
    END IF;

    RETURN TO_CHAR(taux_occupation, 'FM999.9') || '%';
EXCEPTION
    WHEN OTHERS THEN
        RETURN 'Non disponible';
END;
/
```

FONCTIONS PL-SQL:

```
-- Revenu Mensuel
CREATE OR REPLACE FUNCTION calculer_revenu_mensuel RETURN
VARCHAR2
AS
    revenu_total NUMBER;
BEGIN
    SELECT NVL(SUM(c.prix), 0)
    INTO revenu_total
    FROM Réservation res
    JOIN Chambre c ON res.id_chambre = c.id_chambre
    WHERE res.statut = 'Confirmée'
        AND res.date_deb >= TRUNC(SYSDATE, 'MONTH');

    RETURN TO_CHAR(revenu_total, 'FM999999.00') || ' €';
EXCEPTION
    WHEN OTHERS THEN
        RETURN '0.00 €';
END;
/
```

FONCTIONS PL-SQL:

```
-- Nombre Total de Clients
CREATE OR REPLACE FUNCTION calculer_total_clients RETURN
VARCHAR2
AS
    total_clients NUMBER;
BEGIN
    SELECT NVL(SUM(r.nb_personne), 0)
    INTO total_clients
    FROM Réservation r
    WHERE r.statut = 'Confirmée';

    RETURN TO_CHAR(total_clients) || ' clients';
EXCEPTION
    WHEN OTHERS THEN
        RETURN 'Non disponible';
END;
/
```

FONCTIONS PL-SQL:

```
-- Chambres Disponibles
CREATE OR REPLACE FUNCTION calculer_chambres_disponibles RETURN
VARCHAR2
AS
    chambres_disponibles NUMBER;
    total_chambres NUMBER;
BEGIN
    -- Compter les chambres disponibles
    SELECT COUNT(*)
    INTO chambres_disponibles
    FROM Chambre
    WHERE etat = 'Disponible';

    -- Compter le total des chambres
    SELECT COUNT(*)
    INTO total_chambres
    FROM Chambre;

    RETURN chambres_disponibles || '/' || total_chambres;
EXCEPTION
    WHEN OTHERS THEN
        RETURN '0/0';
END;
/
```

LES REQUÊTE SQL UTILISÉE EN CODE JAVA :

Fonction: loadAvailableRooms()

```
SELECT r.room_id, r.room_type, r.price, r.status
      FROM rooms r
      WHERE r.status IN ('Available', 'Pending')
      AND (condition optionnelle sur le type)
      AND vérification des dates de réservation
```

Fonction: showReservationDates()

```
SELECT check_in, check_out, status
      FROM reservations
      WHERE room_id = ?
      AND status IN ('Pending', 'Confirmed')
```

Fonction: showReservationDialog()

```
SELECT COUNT(*) FROM reservations
      WHERE room_id = ?
      AND status != 'Cancelled'
      AND (check_in <= ? AND check_out >= ?)
```

LES REQUÊTE SQL UTILISÉ EN CODE JAVA :

Fonction: showReservationDialog()

```
INSERT INTO reservations (...) VALUES (?, ?, ?, ?, ?, 'Pending')
```

Fonction: cancelBtn

```
UPDATE rooms SET status = 'Available' WHERE  
room_id = ?  
DELETE FROM reservations WHERE reservation_id = ?
```

Fonction: loadRoomsData()

```
SELECT * FROM rooms ORDER BY room_id
```

LES REQUÊTE SQL UTILISÉ EN CODE JAVA :

Fonction: showAddRoomDialog()

```
SELECT COUNT(*) FROM rooms WHERE room_id = ?  
INSERT INTO rooms (...) VALUES (?, ?, ?, ?)
```

Fonction: showReservationDates()

```
SELECT check_in, check_out, status  
FROM reservations  
WHERE room_id = ?  
AND status IN ('Pending', 'Confirmed')
```

Fonction: loadReservationsData()

```
SELECT * FROM reservations ORDER BY reservation_id
```

LES REQUÊTE SQL UTILISÉÈ EN CODE JAVA :

Fonction: editRoom()

- SELECT COUNT(*) FROM rooms WHERE room_id = ?
- INSERT INTO rooms (...) VALUES (?, ?, ?, ?)

Fonction: processReservation()

- UPDATE reservations SET status = ? WHERE reservation_id = ?
- UPDATE rooms SET status = ? WHERE room_id = ?

Fonctions métriques (calculateOccupancyRate)

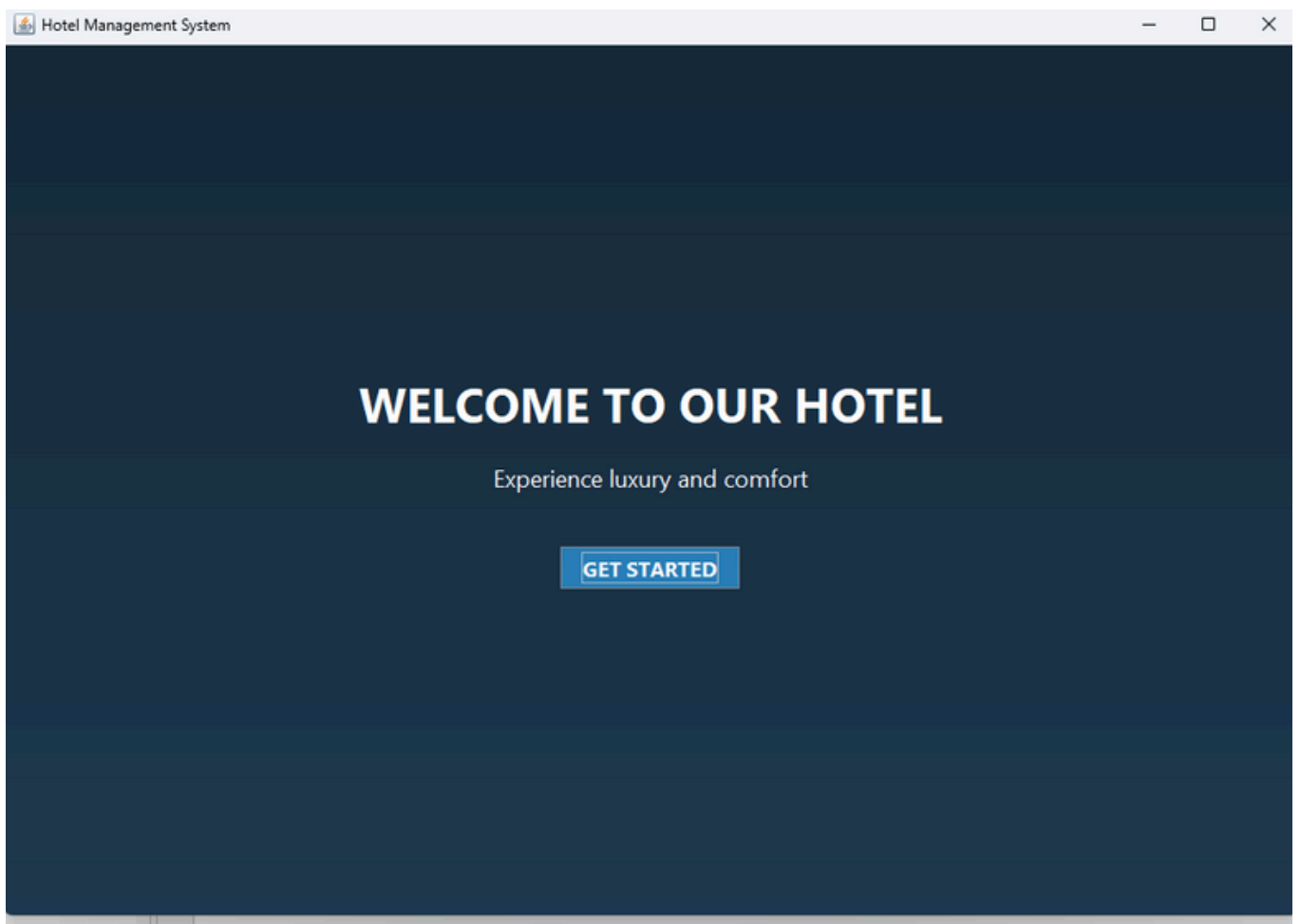
- CALL get_occupancy_rate (procédure stockée)
- CALL get_monthly_revenue (procédure stockée)



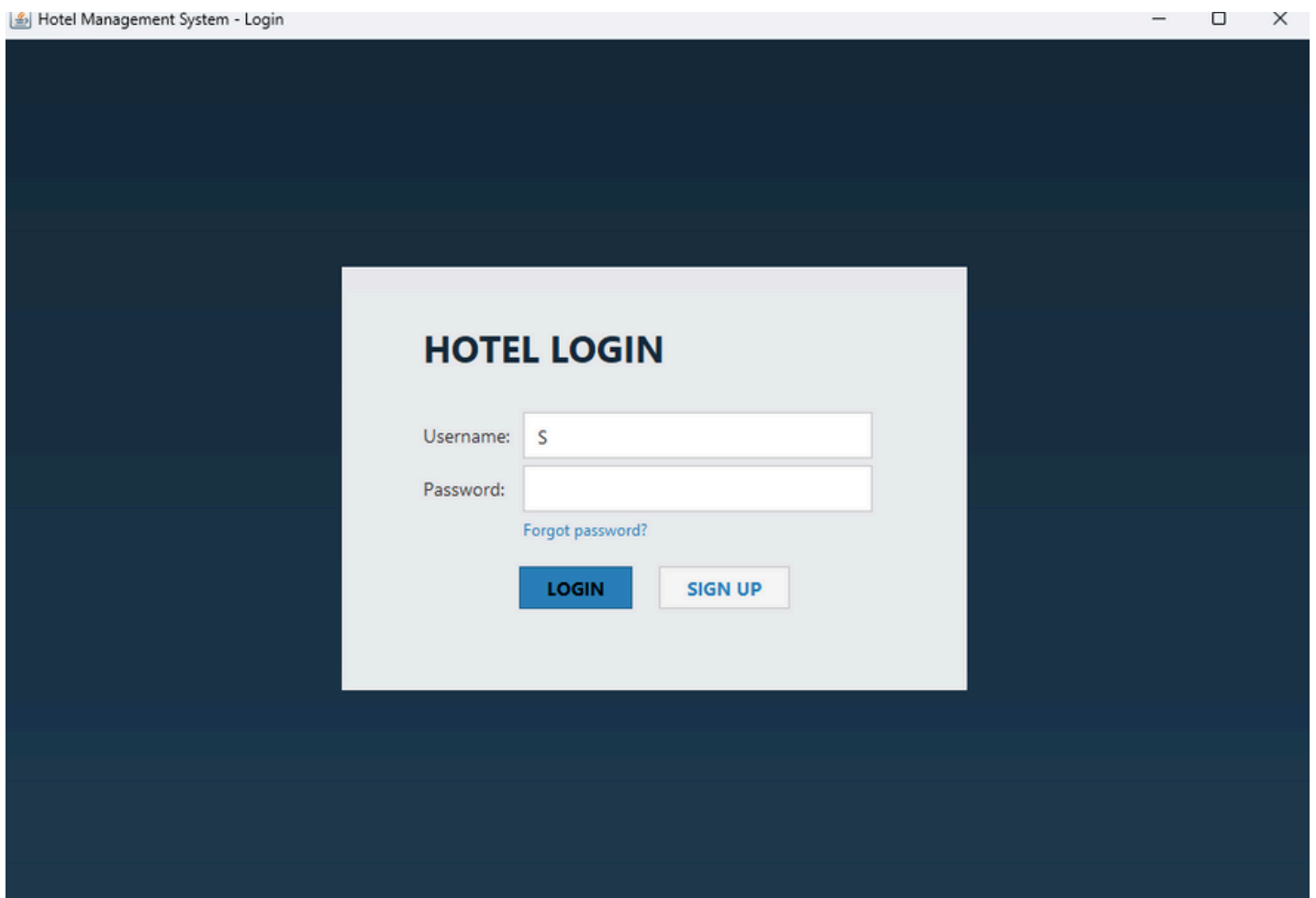
LES INTERFACES :



INTERFACE DE BIENVENUE :



INTERFACE DE LOGIN/SIGNUP:



The screenshot shows a web browser window titled "Hotel Management System - Login". The main content area has a dark blue background. In the center, there is a light gray rectangular box containing the login form. The form has the title "HOTEL LOGIN" in bold. Below the title, there are two input fields: "Username:" with the letter "S" entered, and "Password:". Below the password field is a link that says "Forgot password?". At the bottom of the form, there are two buttons: a blue "LOGIN" button and a white "SIGN UP" button with a blue border.

Hotel Management System - Login

HOTEL LOGIN

Username: S

Password:

[Forgot password?](#)

LOGIN **SIGN UP**



DASHBOARD DU CLIENT :

CLIENT DASHBOARD

User: client

Refresh

Logout

Available Rooms

My Reservations

Check-In Date (YYYY-MM-DD):

yyyy-MM-dd

Check-Out Date (YYYY-MM-DD):

yyyy-MM-dd

Room Type:

All

Search

ID	Type	Price	Status	Reserve	View Reservations
101	Standard	\$100,00	Available	Reserve	View Reservations
203	Standard	\$132,00	Available	Reserve	View Reservations
102	Deluxe	\$150,00	Available	Reserve	View Reservations



DASHBOARD DU CLIENT :

Client Dashboard - Welcome yazid rebei

CLIENT DASHBOARD User: client [Refresh](#) [Logout](#)

Available Rooms **My Reservations**

Check-In Date (YYYY-MM-DD):

Check-Out Date (YYYY-MM-DD):

Room Type:

[Search](#)

ID	Type	Price	Reserve	View Reservations
101	Standard	\$100.00	Reserve	View Reservations
203	Standard	\$132.00	Reserve	View Reservations
102	Deluxe	\$150.00	Reserve	View Reservations

Reservation Dates

Booked dates for Room 101:

From 2025-08-05 to 2025-08-06 (Confirmed)

From 2025-12-15 to 2025-12-18 (Confirmed)

[OK](#)

LE CLIENT PEUT CHERCHER UNE CHAMBRE DISPONIBLE OU
VOIR LES RESERVATIONS AVANT DE RESERVER



DASHBOARD DU CLIENT :

Client Dashboard - Welcome yazid rebei

CLIENT DASHBOARD

User: client

[Refresh](#) [Logout](#)

Available Rooms **My Reservations**

Check-In Date (YYYY-MM-DD):

Check-Out Date (YYYY-MM-DD):

Room Type:

[Search](#)

ID	Type
101	Standard
203	Standard
102	Deluxe

Make Reservation for Room 101

Check-In Date (YYYY-MM-DD):

Check-Out Date (YYYY-MM-DD):

[View Booked Dates](#)

[Submit Reservation](#)

Reserve	View Reservations
Reserve	View Reservations
Reserve	View Reservations
Reserve	View Reservations

LE CLIENT PEUT RESERVER



DASHBOARD DU CLIENT :

Client Dashboard - Welcome yazid rebei

CLIENT DASHBOARD User: client [Refresh](#) [Logout](#)

[Available Rooms](#) [My Reservations](#)

ID	Room	Check-In	Check-Out	Status	Total Price
RES1747404090770	101	2025-09-05	2025-09-10	Pending	\$500,00
RES1747403269996	101	2025-12-15	2025-12-18	Confirmed	\$300,00

[Cancel Selected Reservation](#)

LE CLIENT PEUT VOIR SES RESERVATIONS ET LEURS COUTS
TOTAUX ET PEUT ANNULER UNE RESERVATION QUAND ELLE EST
EN ATTENTE



DASHBOARD D'ADMIN :

Tableau de Bord Admin - rayen bouajila

GESTION HÔTELLIÈRE - ADMIN Connecté en tant que: admin [Déconnexion](#)

Chambres Réservations Comptes Statistiques

ID	Type	Prix	Statut	Modifier	Supprimer	Voir Réservations
101	Standard	\$100.00	Pending	Modifier	Supprimer	Voir Réservations
102	Deluxe	\$150.00	Available	Modifier	Supprimer	Voir Réservations
203	Standard	\$132.00	Available	Modifier	Supprimer	Voir Réservations

+ Ajouter Chambre

L'ADMIN PEUT GERER LES CHAMBRES : AJOUTER OU MODIFIER
OU VOIR LES RESERVATIONS



DASHBOARD D'ADMIN :

Navigation: [Chambres](#) [Réservations](#) **[Comptes](#)** [Statistiques](#)

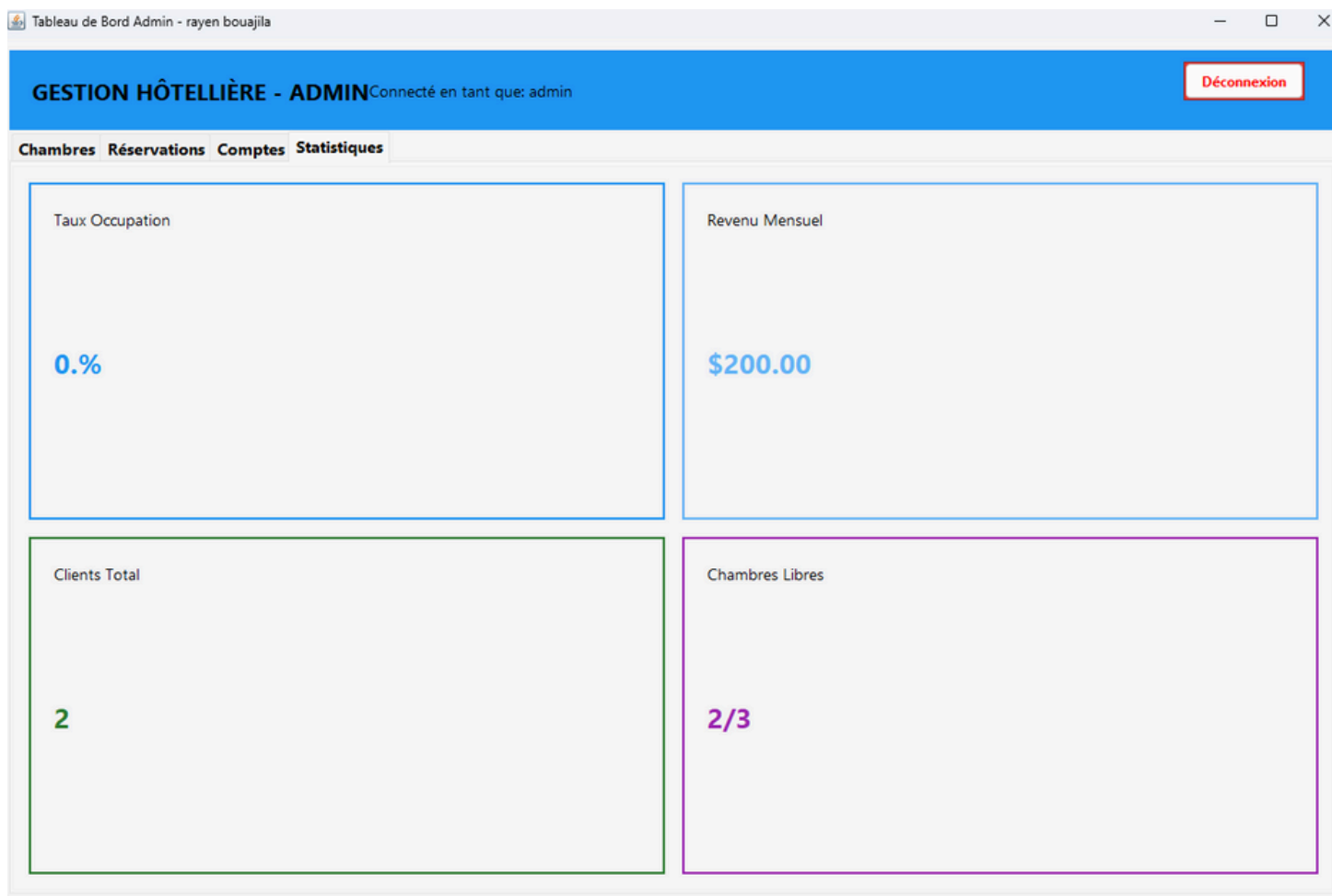
Username	Full Name	Role	Change Role	Delete
admin	rayen bouajila	admin		
ahmed	ahmed	client	Change Role	Delete
client	yazid rebei	client	Change Role	Delete

+ Add User

L'ADMIN PEUT GERER LES COMPTES DE CLIENTS : AJOUTER OU MODIFIER LE ROLE OU VOIR LES CLIENTS



DASHBOARD D'ADMIN :



L'ADMIN PEUT VOIR LES STATISTIQUES



DASHBOARD D'ADMIN :

Tableau de Bord Admin - rayen bouajila

GESTION HÔTELLIÈRE - ADMIN Connecté en tant que: admin [Déconnexion](#)

Chambres Réservations Comptes Statistiques

ID	Client	Chambre	Arrivée	Départ	Statut	Confirmer	Annuler	Supprimer
RES1747403269...	yazid rebei	101	2025-12-15	2025-12-18	Confirmed		Annuler	
RES1747403822...	ahmed	101	2025-08-05	2025-08-06	Confirmed		Annuler	
RES1747404090...	yazid rebei	101	2025-09-05	2025-09-10	Pending	Confirmer	Annuler	

[Supprimer Réservations Annulées](#)

L'ADMIN PEUT GERER LES DEMANDES DE RESERVATIONS PAR LES CONFIRMER OU LES ANNULER OU SUPPRIMER UNE RESERVATION DEJA ANNULE DE LA BASE DE DONNES





**MERCI DE VOTRE
ATTENTION**

